Lab 10 — Memory Management and Page Replacement

Raj Pratap Singh Gurjar | 2401MC21 | Operating Systems Lab

Base: xv6-public commit eeb7b415dbcb12cc362d0783e41c3d1f44066b17

Overview

This assignment adds physical page counters, virtual-to-physical translation, page permission inspection and protection, and three page replacement simulations. The syscall changes are split into four patches and apply in question order to the base commit above.

Q1 — Physical memory tracking

The allocator keeps a free-list count alongside runtime allocation and free totals. The initialization calls that populate the free list are not included in those runtime totals. The `memstats()` syscall prints allocated pages, freed pages, outstanding pages, and currently available pages. `memtest` touches eight pages obtained with `sbrk()`, checks the counts, and releases the block. Available memory fell by eight pages and returned to its earlier count.

Q2 — Virtual-to-physical translation

The `vtop()` syscall uses `walkpgdir()` to locate a present, user-accessible mapping. It combines the physical frame address with the byte offset from the virtual address, and returns -1 for an address outside the process range or without a valid mapping. `vttest` checks a heap address at offset 37, a code mapping, and an unmapped address.

Q3 — Page information and protection

`pageinfo()` displays the frame number and P/W/U permission bits. `setreadonly()` clears `PTE_W` and reloads the current page directory so the CPU drops any cached writable translation. The test confirms W changes from 1 to 0, then tries the write in a child process. xv6 reports a protection fault and kills the child; the parent remains alive to report the result.

Q4 — Page replacement

FIFO replaces the oldest loaded page, LRU replaces the page least recently referenced, and Optimal replaces the page whose next use is farthest in the future. The program prints the frame contents and hit or fault result after every reference for three frames, then calculates the totals for three, four, and five frames.

Reference string: `7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1`

Comparison for the assigned 20-page reference string

Frames	Policy	Faults	Hits	Hit ratio	Fault ratio
3	FIFO	15	5	0.250	0.750
3	LRU	12	8	0.400	0.600
3	Optimal	9	11	0.550	0.450
4	FIFO	10	10	0.500	0.500
4	LRU	8	12	0.600	0.400
4	Optimal	8	12	0.600	0.400
5	FIFO	9	11	0.550	0.450
5	LRU	7	13	0.650	0.350
5	Optimal	7	13	0.650	0.350

Optimal has the fewest faults with three frames. At four and five frames, LRU ties Optimal on this trace. FIFO has the highest fault count in all three frame configurations. Hit and fault ratios use 20 as the denominator.

Run details

All four programs built and ran on xv6-public with i686-elf-gcc, qemu-system-i386, 512 MB RAM, and one CPU. The full serial output is in `output_log.txt`; one terminal capture for each question is in `screenshots/`.